

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Mock Interview

Course Code:	ITA0517	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	Yoga Madha A	Reg. No.:	192425058
Section / Batch:	B	Date:	24-08-2026

Code of Conduct

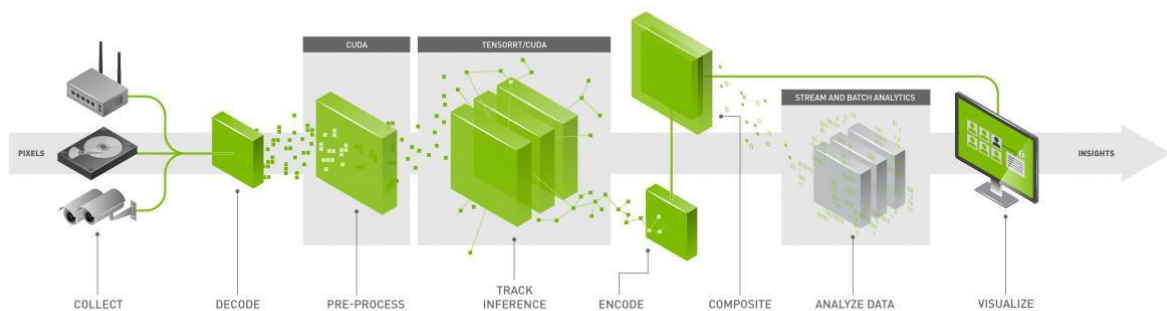
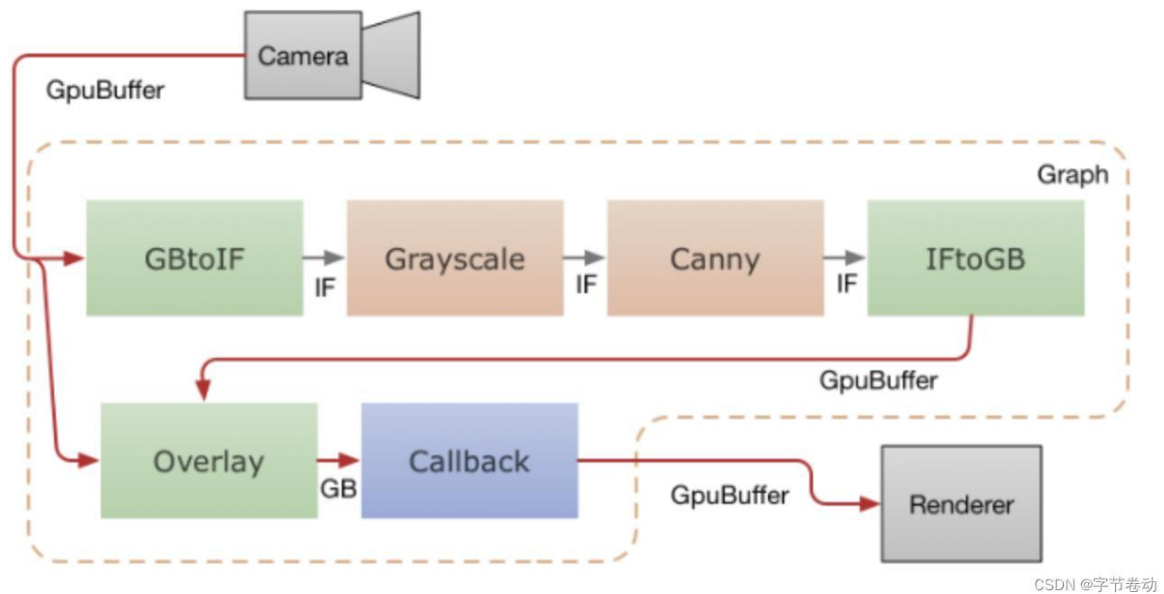
I Yoga Madha A-192425058 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	10	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

22. Real-Time Image Display Optimization



Introduction

A real-time image application must capture, process, and display images with very low delay. High latency can occur when the application performs unnecessary image processing, uses large image sizes, copies images repeatedly, or waits for each processing step to finish before displaying the next frame.

The main objective is to identify the bottlenecks and redesign the pipeline so that images are displayed smoothly and quickly.

Existing Pipeline

Camera/Image Input → Image Capture → Image Decoding → Preprocessing → Image Processing → Memory Copy → Display

If every stage is performed sequentially, the display may become slow and the application may show old frames.

Major Bottlenecks

1. High Image Resolution

Processing very large images requires more CPU and memory.

Example:

A 4K image contains much more data than a 720p image. Processing every pixel increases processing time.

Solution: Resize the image to the required display resolution before processing.

2. Slow Image Processing

Complex operations such as filtering, edge detection, object detection, and feature extraction can increase latency.

Solution: Remove unnecessary operations and use optimized OpenCV functions.

3. Repeated Memory Copying

Copying images between different buffers consumes time and memory bandwidth.

Solution: Use shared buffers or reuse existing memory whenever possible.

4. Sequential Processing

If capture, processing, and display happen one after another, the next frame cannot be captured until the current frame is completely processed.

Solution: Use parallel processing or separate threads for capture, processing, and display.

5. Large Image Format

Images with large file sizes require more time for decoding and transferring.

Solution: Use efficient image formats and appropriate resolutions.

6. CPU Overload

Running all operations on the CPU can cause high utilization and frame delays.

Solution: Use GPU acceleration when available for computationally intensive operations.

7. Display Synchronization

Waiting for every frame to be displayed can create a queue of old frames.

Solution: Drop outdated frames and always display the latest available frame.

Redesigned Fast Pipeline

Camera → Frame Capture → Resize → Preprocessing → Parallel Processing → Latest Frame Buffer → Display

Step 1: Fast Frame Capture

Capture frames continuously using a dedicated capture thread.

This prevents the camera from waiting for image processing to finish.

Step 2: Resize Early

Reduce the image size immediately after capture.

For example:

$1920 \times 1080 \rightarrow 640 \times 360$

This significantly reduces the number of pixels that need to be processed.

Step 3: Efficient Preprocessing

Perform only necessary operations such as:

- Color conversion
- Noise reduction
- Resizing
- Normalization

Avoid unnecessary repeated operations.

Step 4: Parallel Processing

Divide the pipeline into separate tasks:

- Thread 1 – Capture
- Thread 2 – Image Processing
- Thread 3 – Display

This allows different frames to be processed simultaneously.

Step 5: Use Latest-Frame Strategy

Instead of storing every frame in a large queue, maintain only the latest frame.

If processing is slower than the camera, old frames are discarded.

This reduces **display latency**.

Step 6: GPU Acceleration

For computationally intensive operations, use GPU acceleration where available.

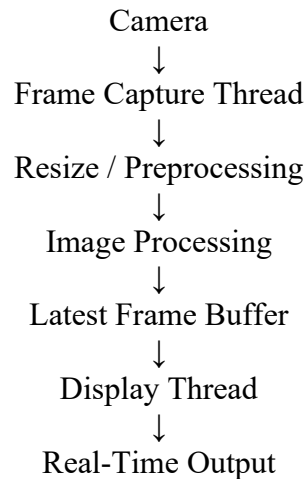
GPU processing can be useful for:

- Deep learning inference
- Image filtering
- Object detection
- Feature extraction
- Large matrix operations

Step 7: Efficient Display

Display the processed frame immediately using an optimized display method.
Avoid unnecessary conversion and copying before displaying the image.

Optimized Pipeline



Pseudocode

START

Initialize camera

Initialize processing thread

Initialize display

WHILE application is running:

 Capture latest frame

 IF frame is available:

 Resize frame

 Apply required preprocessing

 Process frame

 Store processed frame in latest-frame buffer

 Display latest processed frame

 IF user presses exit key:

BREAK

Release camera
Release memory
Close display

END

Performance Improvements

Problem	Optimization
Large images	Resize before processing
Slow processing	Use optimized algorithms
Repeated memory copies	Reuse buffers
Sequential execution	Multithreading
CPU overload	GPU acceleration
Old frames	Drop outdated frames
Large queues	Use latest-frame buffer
Unnecessary operations	Remove redundant processing
Slow decoding	Use efficient image formats
Display delay	Display latest processed frame

Expected Result

The redesigned pipeline provides:

- Lower image display latency
- Higher FPS
- Smooth real-time display
- Lower CPU and memory usage
- Better responsiveness
- Reduced frame backlog
- Faster image processing
- Improved user experience

Conclusion

The major cause of high latency is usually a combination of **large image sizes, sequential processing, unnecessary memory copying, complex processing, and frame buffering**. The pipeline can be optimized by **resizing images early, using efficient algorithms, parallelizing capture/processing/display, reusing memory, using GPU acceleration, and displaying only the latest frame**. This produces a faster and smoother real-time image application.